Getting Started

Introduction to computer programming

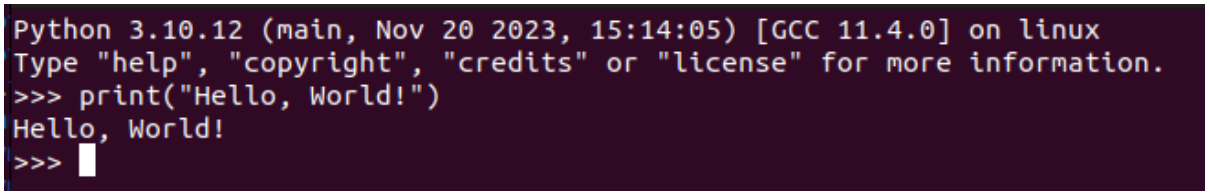
Management and Artificial Intelligence

How and Where

How can we run Python code?

Two approaches:

1. Interactive shell:

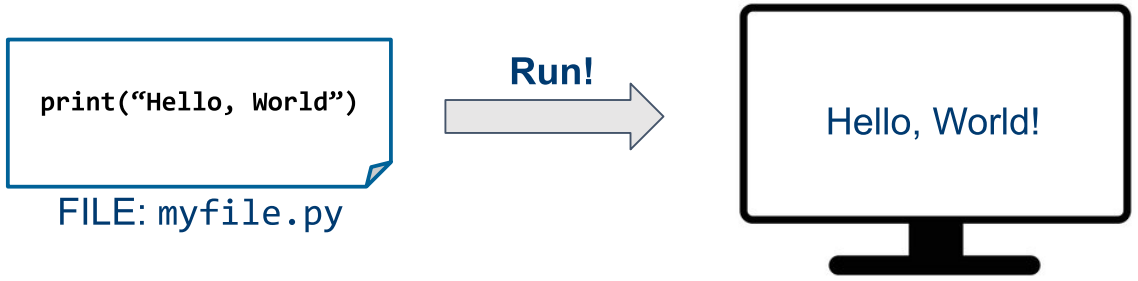


- workflow:
    - start the interactive Python shell
    - write one line of code in the prompt after the symbols `>>>`
    - push enter to execute it and get the results
    - repeat as musch as needed
- not very user-friendly
- appropriate only for quick tests
- nothing is saved: you will lose your code when exiting the shell!

How can we run Python code? (cont'd)

Two approaches:

2. Write the code into a textual file and then execute it:



- workflow:
  - write the code into a textual file with extension `.py`
  - execute the file with the Python VM
- This is what you want to do!
- Still *iterative* approach: write the code, check if it works, refine the code, ...

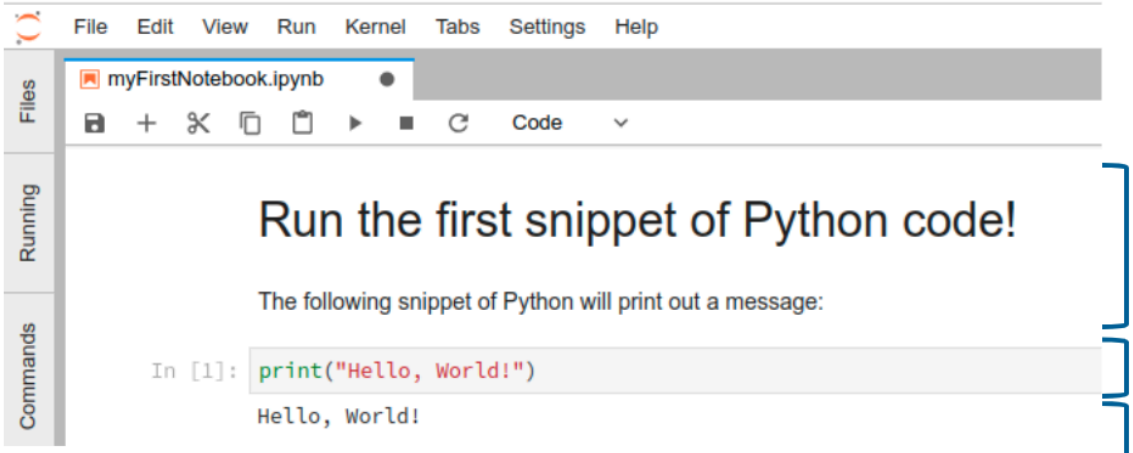
The code can be very cryptic...
Can we add some notes around it?

It could be hard to remember why we have written a piece of code, what are its effects and downsides.

It is often convenient to organize our code into "sections" and then add some text to describe what the code is expected to do. This is called:

Jupyter Notebook

Using the environment JupyterLab we can create a new notebook:



Text section describing what we is relevant for us (humans!)

snippet of code

result of the code (after running it with the play button)

Where do we run our code/notebooks?

1. Local setup: your computer!

- you have to install Python and all the other bits
- limited by your computational power

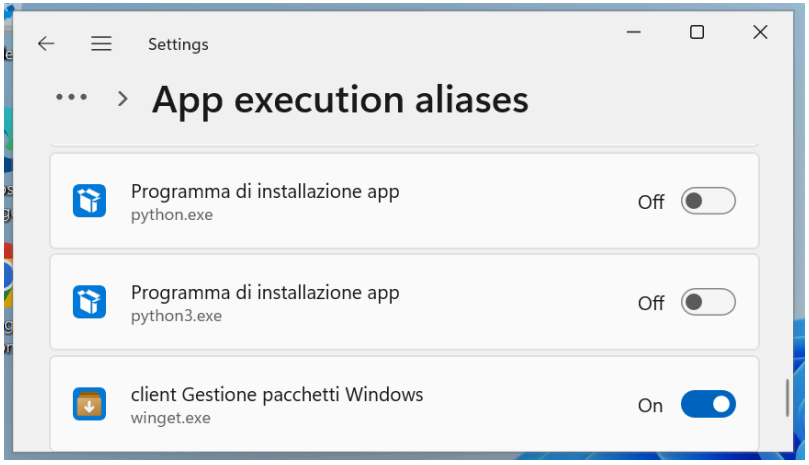
2. Cloud setup:

- no need to install everything
- limited by your money

Local Setup

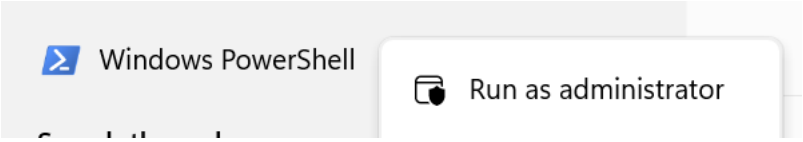
[**Only on Windows 11**]
Disable Windows Store Aliases

1. From the start menu, search and then open *Manage app execution aliases* (*Alias di esecuzione delle app*)
2. Disables aleases for *python.exe* and *python3.exe*:

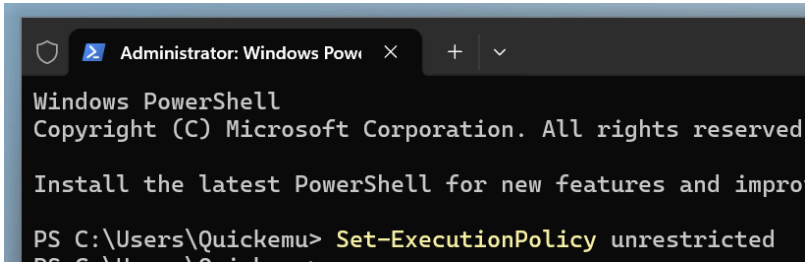


[Only on Windows 11]
Enable script cmdlet

1. From the start menu, search *Windows Powershell*, then using the right-click select *Run as administrator*:

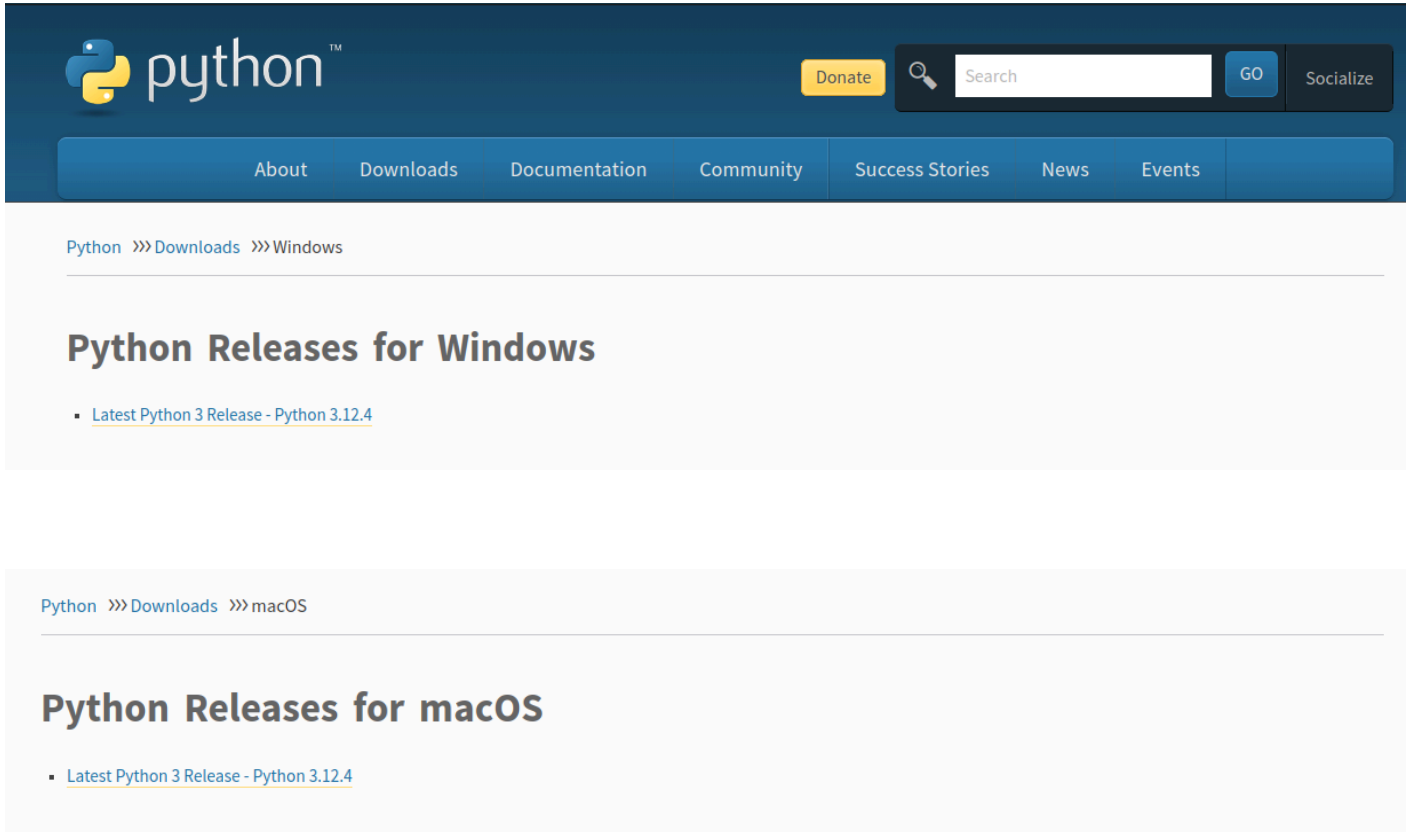


2. Type `Set-ExecutionPolicy unrestricted`, then run it and reply *y* (or "s" if your Windows is in italian) :



Download Python

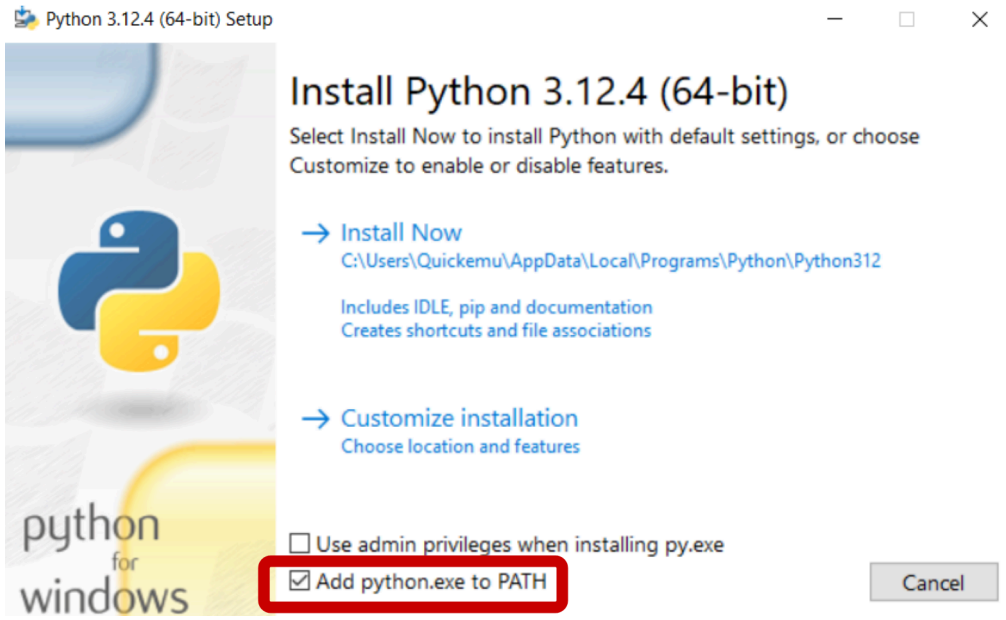
Visit: https://www.python.org/



Launch the installer

Double click on the downloaded file, e.g., on Windows:

1. Enable ***Add python.exe to PATH***
2. Click *Install now*

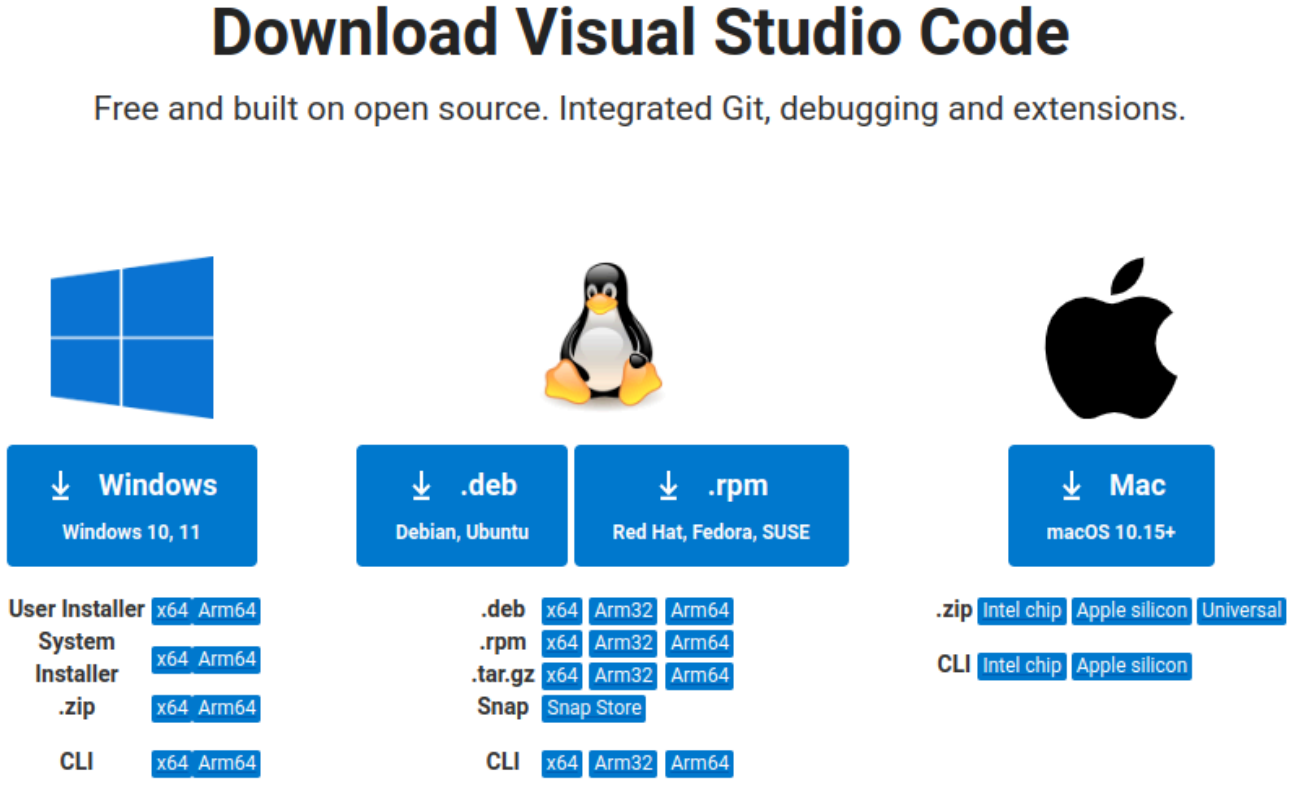


Launch the installer (cont'd)

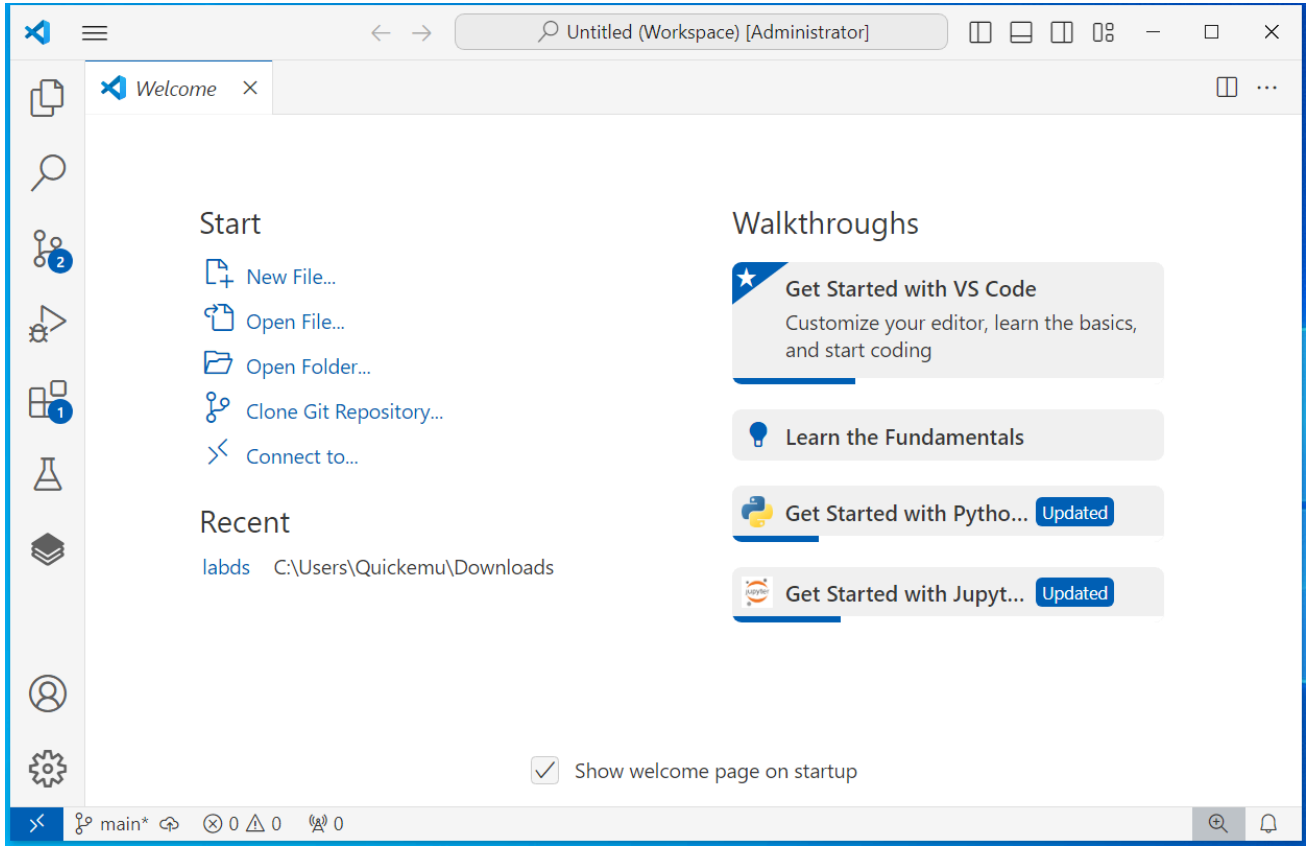
Similarly on Mac OS X:

Install a (good) code editor: **Visual Studio Code**

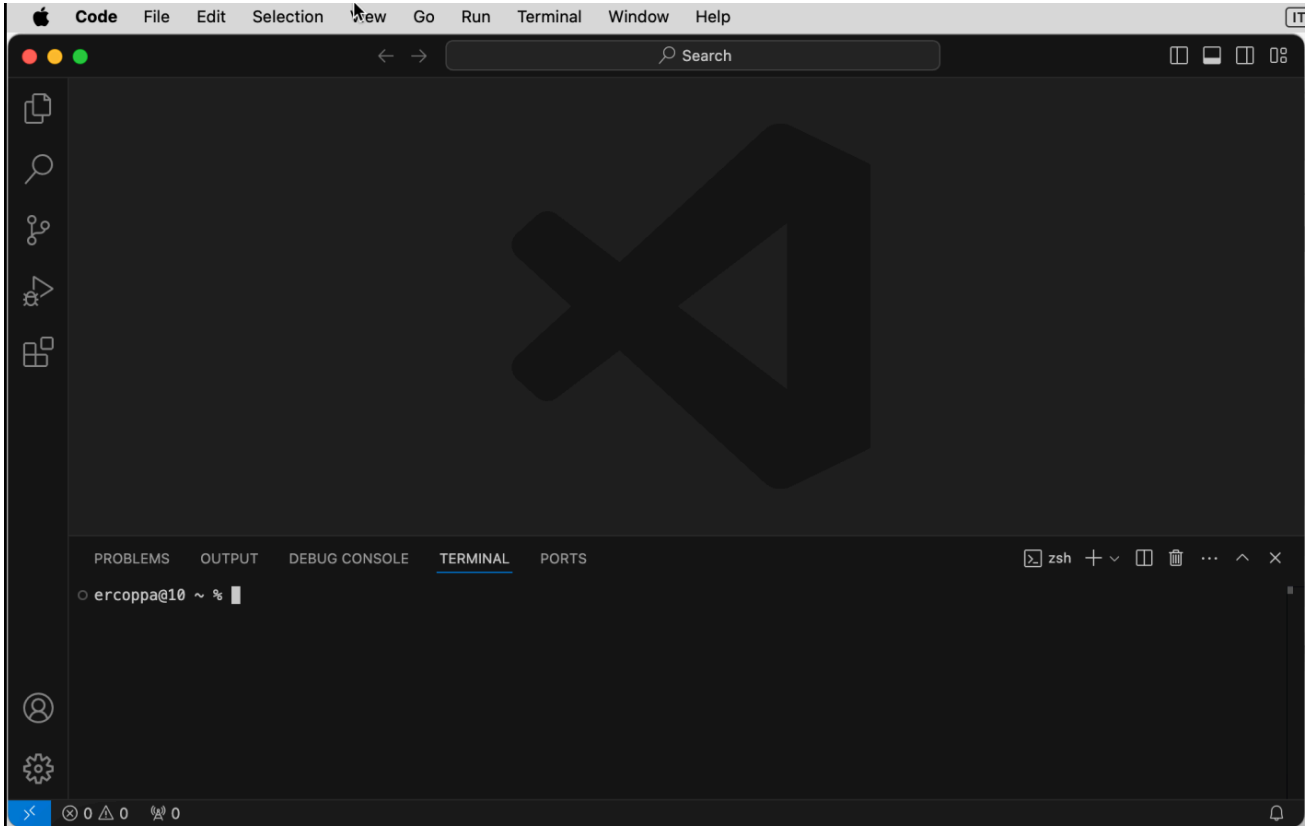
1. Visit: https://code.visualstudio.com/Download
2. Download the latest version for your OS



Start **Visual Studio Code** (Windows)

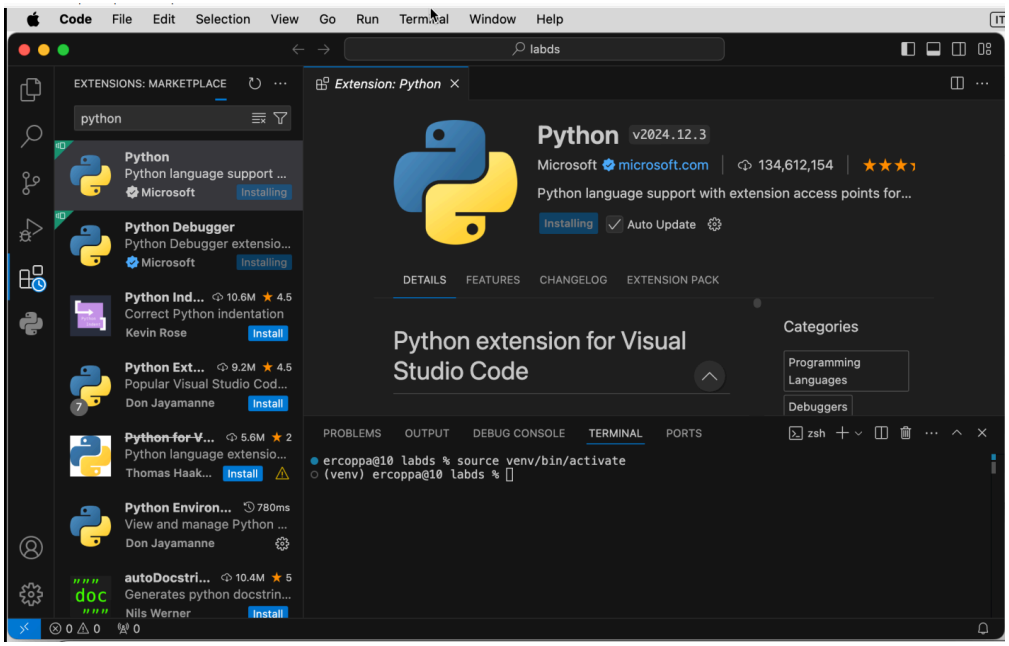


Start **Visual Studio Code** (Mac OS X)



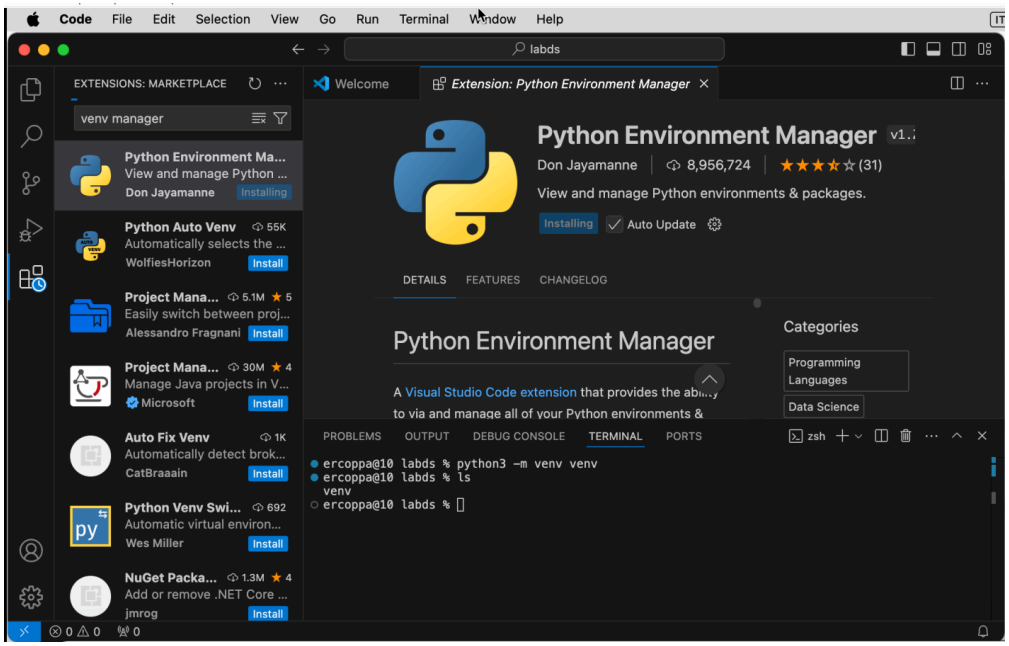
VSCode extension: Python integration

1. From the side bar, open the *Extension* tab
2. Search for *Python*
3. Install the extension



VSCode extension: Python Environment Manager

1. From the side bar, open the *Extension* tab
2. Search for *Python Environment Manager*
3. Install the extension



VSCode extension: Jupyter

1. From the side bar, open the *Extension* tab
2. Search for *Jupyter*
3. Install the extension

Where to install Python packages from the community

We can install Python packages from the community following two strategies:

- *System-wide*: this may create conflicts between different projects using Python.
    - pros: you need to install a package only once for all projects
    - cons: one project may break due to a package installed/updated for another project

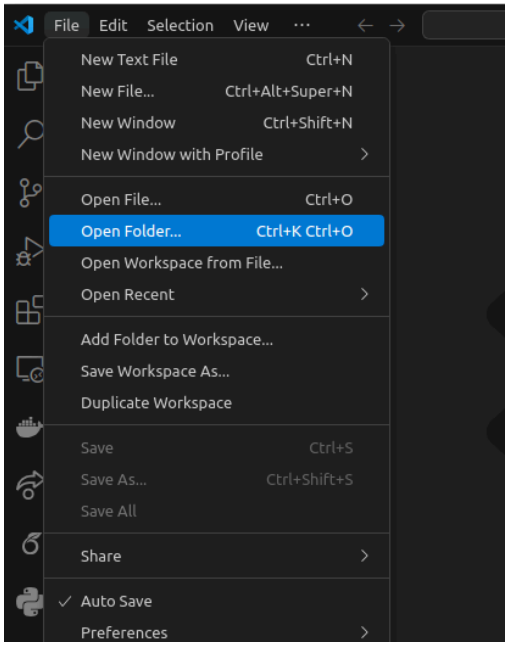
  It is now the deprecated approach.

- *Project-wide*: each project has its own *Virtual Environment*.
    - pros: no conflicts across packages of different projects
    - cons: a package must be installed again for each project

  This is nowadays the suggested approach and we will follow it.

Create a new virtual environment

1. Create a new folder (e.g., `labds` within `Desktop` folder)
2. Open the folder from VSCode:



3. Start the terminal: *View > Terminal*
4. Type in the terminal:

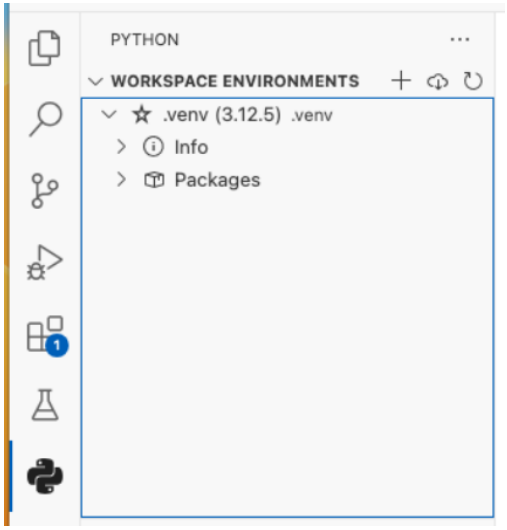
   ```
   python -m venv .venv
   ```

   or:

   ```
   python3 -m venv .venv
   ```

Pick the default virtual environment

1. From the side bar, open the Python logo
2. Click on the *star* to select the virtual environment



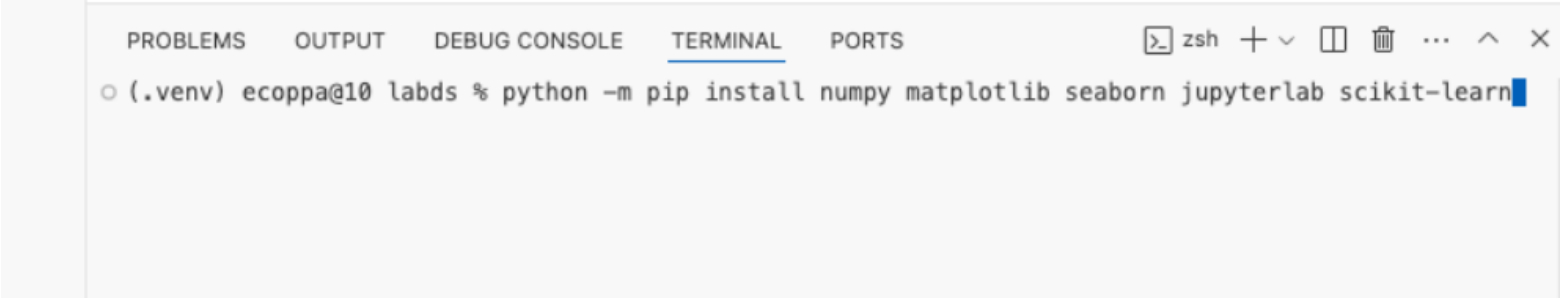
3. Close VSCode and then re-open your project folder in VSCode

Installation of Python packages in the virtual environment

We will use some extra Python bits from the community, including: `numpy`, `matplotlib`, `seaborn`, `scikit-learn`, and `jupyterlab`.

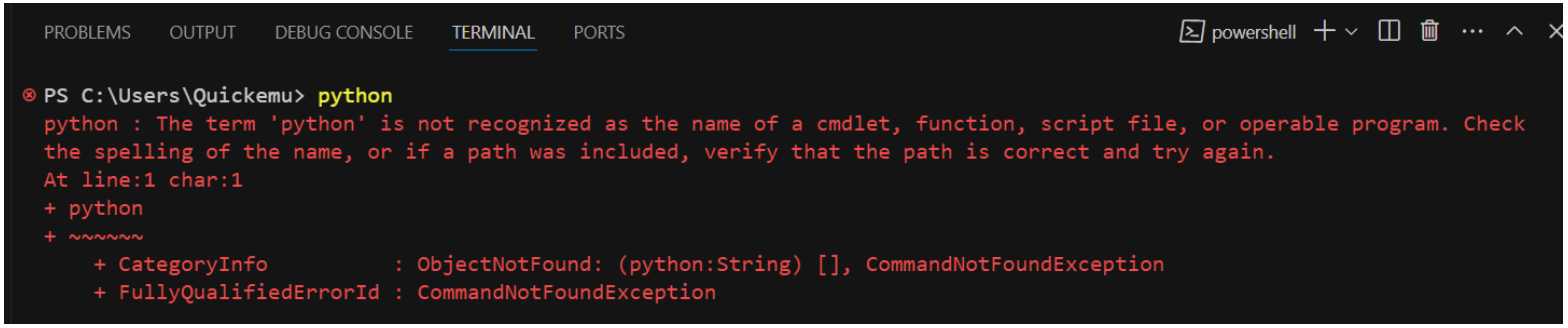
From VSCode:

1. Start a new terminal using the menu: *Terminal > New Terminal*
2. `python -m pip install numpy matplotlib seaborn scikit-learn jupyterlab`



Troubleshooting on Windows: `python is not recognized`

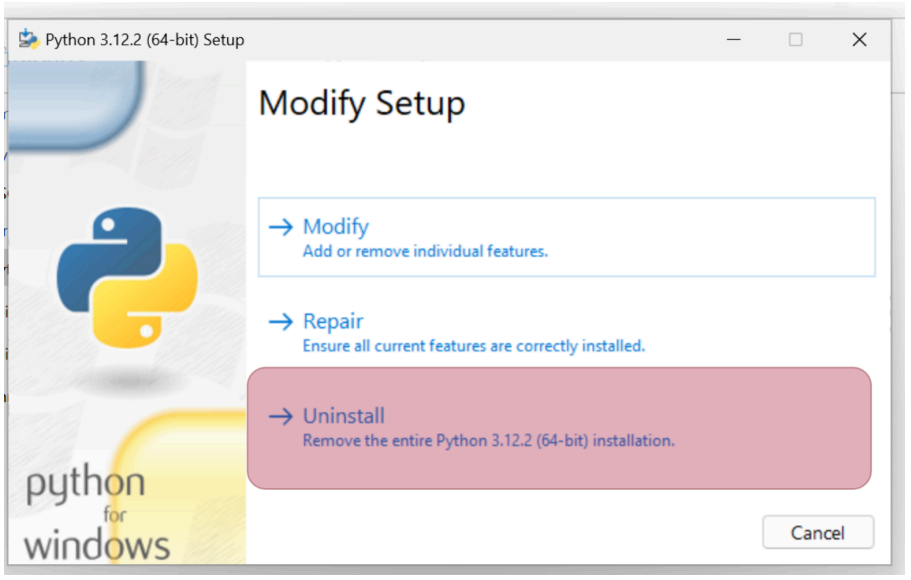
If, when when running `python` in the terminal of VSCode, you get the following error:



Then, you have to:

Troubleshooting on Windows: `python is not recognized` (cont'd)

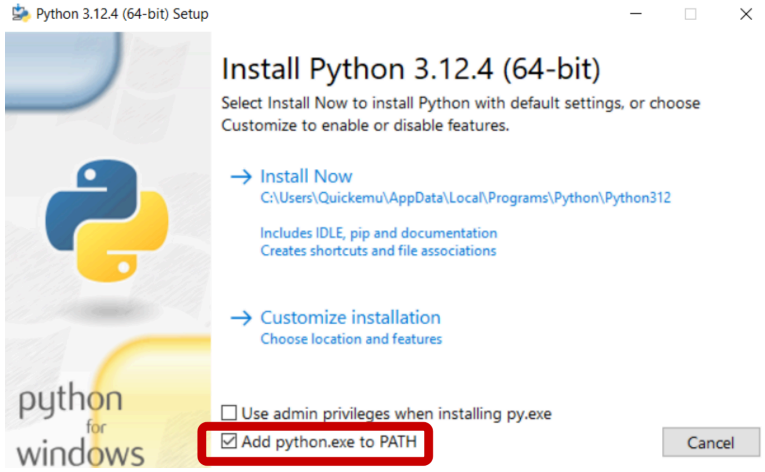
1. Uninstall Python: run again the installer of Python and select *Uninstall*:



Troubleshooting on Windows: `python is not recognized` (cont'd)

2. Start the Python installer

3. Select *Add to path* at the bottom



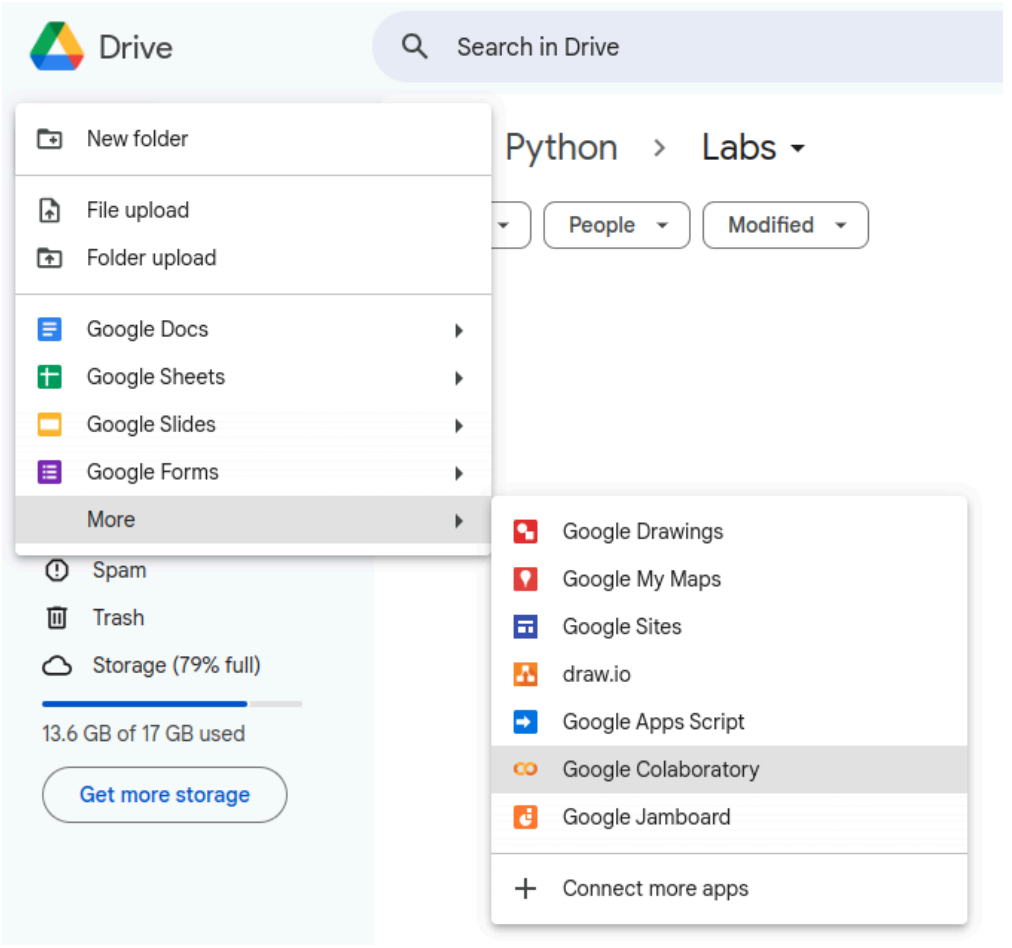
4. Proceed with *Install now*

Cloud Setup

Google Colab, i.e., the lazy approach

- Platform from Google
- Google Account is required
- Free of charge to use for the basic plan
- Write code in the browser (no extra is needed)
- Code executed on Google cloud servers
- Low/medium performance, privacy issues
- No need to install anything

From **Google Drive**



>

Environments

Environment: local setup with VSCode

Use VSCode:

- Quickly write and execute python files (`.py`)
- Quick access to the local terminal
- Write and execute notebooks (`.ipynb`). Some advanced features of JupyterLab (that we do not need) may have limited support.
- Good editor with several extensions (integrations with third-party services)
- Everything runs on your machine

This is the environment that we suggest you to use.

Environment: local setup with JupyerLab

Use JupyterLab:

- Start it from any terminal (e.g., from VSCode) with: `python -m jupyterlab`
- Write and execute notebooks (`.ipynb`) with support even for advanced and weird features.
- Basic editor
- Everything runs on your machine

We suggest to use JupterLab only when you need to run advanced Jupyter notebooks obtained from the external world

Environment: cloud setup with Colab

Use Google Colab:

- You can work from any computer. Even your tablet (although, it is a bit inconvenient!)
- Write and execute on the cloud notebooks (.pynb). Some advanced features of JupyterLab (that we do not need) may have limited support.
- Basic editor
- Everything must be saved on Google Drive
- Not adequate in future courses where you need a bit of computational power (assuming you do not want to pay Google).

We suggest to use Colab only if you issues with your local setup